

Examen Final: "La Carrera de Colapinto en la F1"

Ejercicio 1: Listas por Comprensión

Tema: Clasificación de las posiciones de Colapinto.

Se tiene una lista con las posiciones obtenidas por Colapinto en sus últimas carreras. Filtra, usando listas por comprensión, las posiciones donde terminó en el podio (1, 2, 3).

```
posiciones = [1, 5, 2, 8, 3, 4, 10, 2, 1]
# Tu código aquí
```

Ejercicio 2: Clases y Objetos

Tema: Modelo de un auto de F1.

Implementa una clase `AutoF1` que tenga atributos como `equipo`, `potencia_motor`, y `neumaticos`. Agrega un método para calcular el tiempo estimado de vuelta dada la potencia y la adherencia de los neumáticos.

```
# Clase AutoF1
class AutoF1:
    def __init__(self, equipo, potencia_motor, neumaticos):
        pass
    # Método para calcular tiempo
```

Ejercicio 3: Teoría (Scope y Tiempo de Vida de las Variables)

Tema: Variables en Python.

1. Explica qué son las variables locales y globales en Python, y da un ejemplo de cada una.
2. ¿Qué sucede si intentas modificar una variable global dentro de una función sin declararla explícitamente como `global`?
3. Define el concepto de "tiempo de vida" de una variable y cómo afecta al diseño de programas.

Ejercicio 4: Recursividad

Tema: Puntos acumulados.

Colapinto gana puntos según su posición. Si es 1° obtiene 25 puntos, 2° 18 puntos, 3° 15 puntos, 4° 12 puntos, 5° 10 puntos, 6° 8 puntos, 7° 6 puntos, 8° 4 puntos, 9° 2 puntos y 10° 1 puntos. Escribe una función recursiva que calcule el total de puntos acumulados en una lista de posiciones.

```
def calcular_puntos(posiciones, n):  
    pass
```

Ejercicio 5: Pilas

Tema: Gestión de paradas en boxes.

Cada vez que Colapinto hace una parada, su equipo registra las tareas en una pila (ej., cambiar neumáticos, ajustar alerón). Implementa la funcionalidad de una pila con listas enlazadas para simular este proceso.

```
class Nodo:  
    def __init__(self, dato):  
        pass  
  
class Pila:  
    def __init__(self):  
        pass  
    # Métodos push y pop
```

Ejercicio 6: Ordenamiento

Tema: Tiempos de vuelta.

Ordena una lista de tiempos de vuelta de menor a mayor utilizando el algoritmo de selección.

```
def ordenar_tiempos(tiempos):  
    pass
```

Ejercicio 7: Manejo de Archivos

Tema: Registro de carreras.

Escribe un programa que lea un archivo con los resultados de las carreras (una por línea: carrera, posición) y calcule cuántas veces Colapinto quedó en cada posición.

```
# Ejemplo de archivo: carrera1,1\n carrera2,2\n ...  
def procesar_resultados(archivo):  
    pass
```

Ejercicio 8: Búsqueda Binaria

Tema: Análisis de tiempos.

Dado un registro ordenado de tiempos de vuelta, implementa búsqueda binaria para verificar si Colapinto hizo una vuelta en un tiempo específico.

```
def busqueda_binaria(tiempos, tiempo_buscado):  
    pass
```

Ejercicio 9: Teoría (Complejidad Computacional)

Tema: Análisis de algoritmos de ordenamiento.

1. Describe cómo funciona el algoritmo de selección para ordenar elementos y su complejidad temporal en el peor caso.
 2. ¿Qué ventajas tiene el algoritmo de ordenamiento rápido (QuickSort) respecto al de selección?
-

Ejercicio 10: Teoría (Estructuras de Datos)

Tema: Comparación de estructuras de datos.

1. Explica las diferencias entre una pila y una cola, y da un ejemplo práctico del uso de cada una.
2. ¿Por qué las listas enlazadas son más eficientes que las listas tradicionales para ciertas operaciones?